

Session 3: Search

Last session:

- Credit assignment as an information process
- Two ways of thinking about information processing:
 - **Syntactic information**: bits structured into bit strings
 - **Semantic information**: physical symbols structured into lists

This session:

- Credit assignment as a search process
- Search = strategies for making information processing more **efficient**

Visualizing the basic idea of search

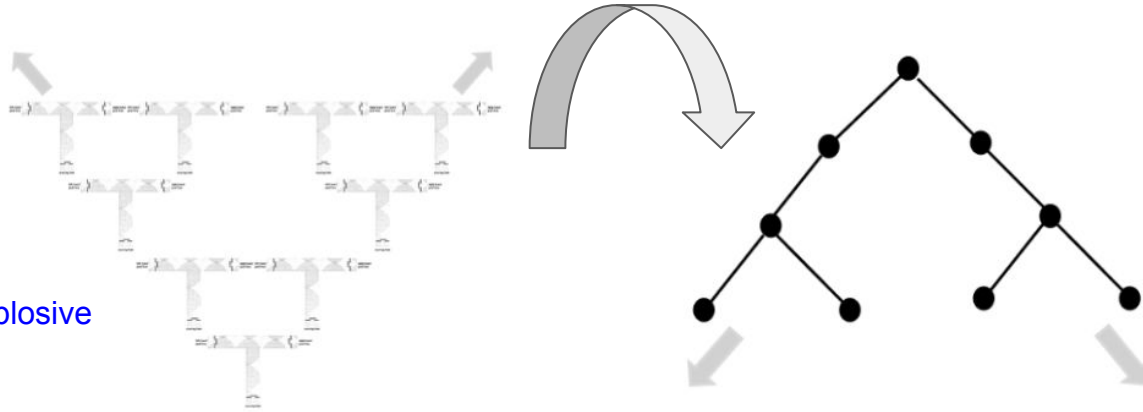
Visualizing the basic idea of search

T-maze

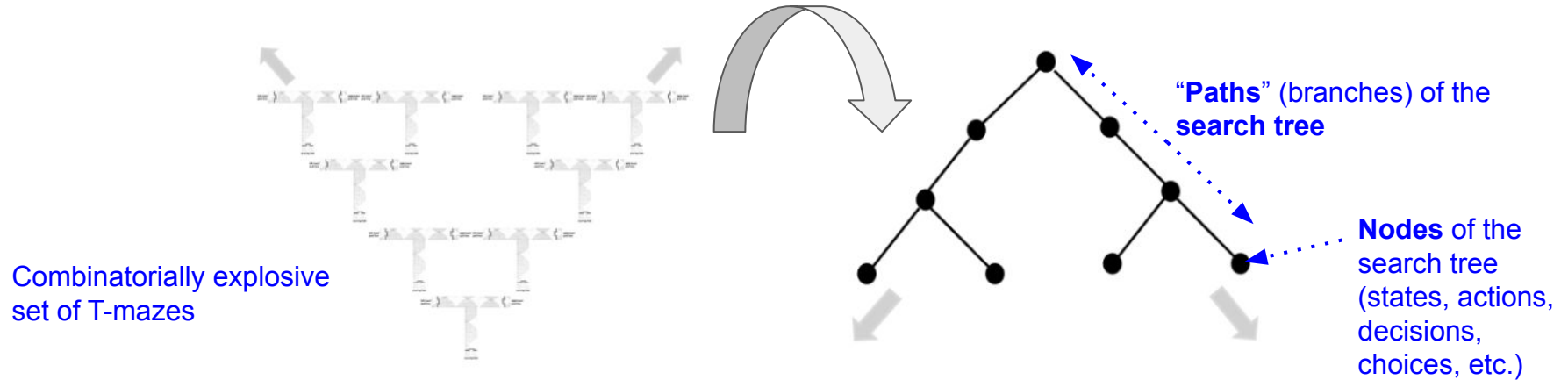


Visualizing the basic idea of search

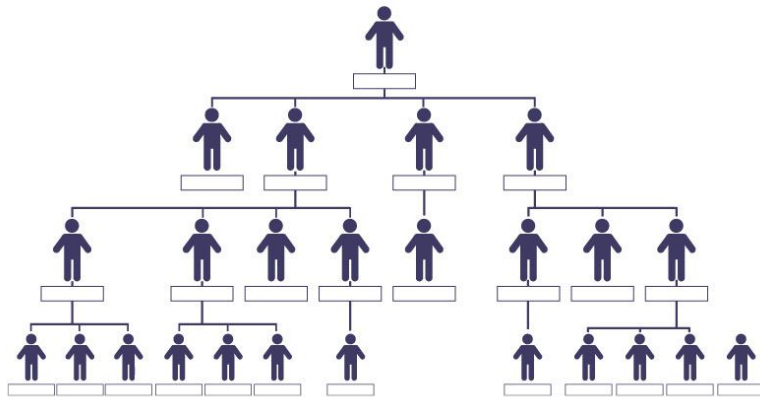
Combinatorially explosive
set of T-mazes



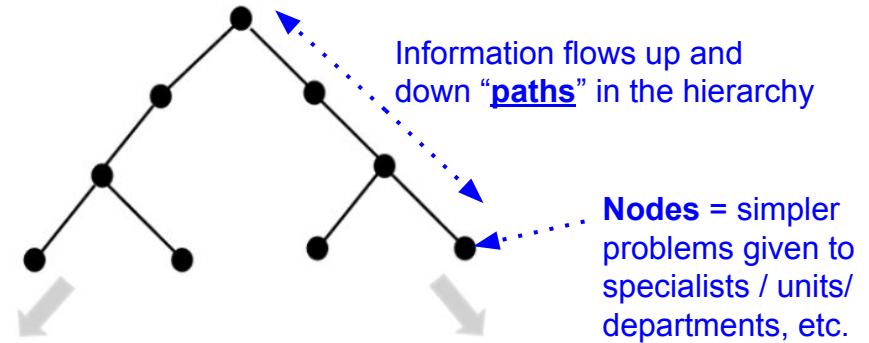
Visualizing the basic idea of search



Visualizing the basic idea of search

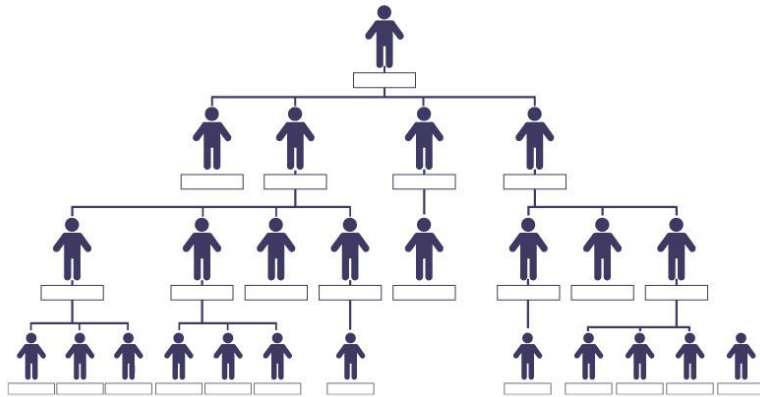


Hierarchical division of labor

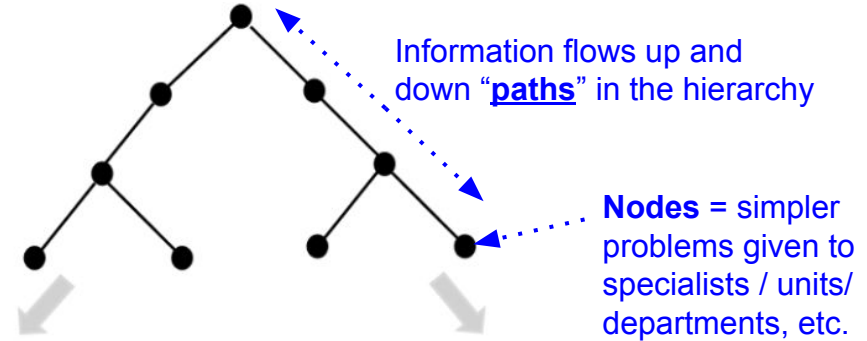


If a problem is too complex, divide it into simpler ones.

Search is based on learning, and learning to select, **plans**



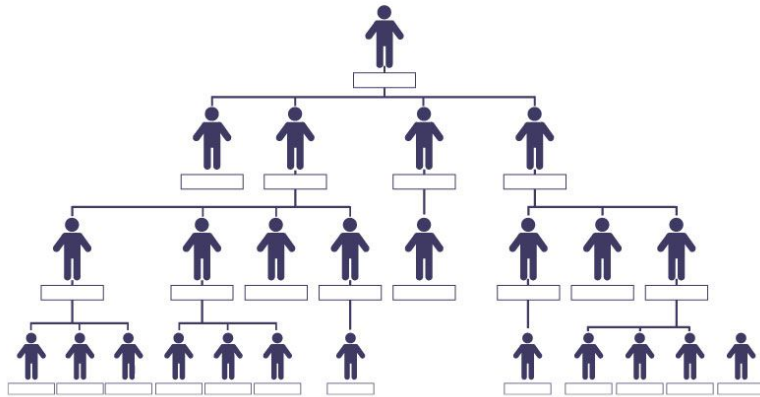
Hierarchical division of labor



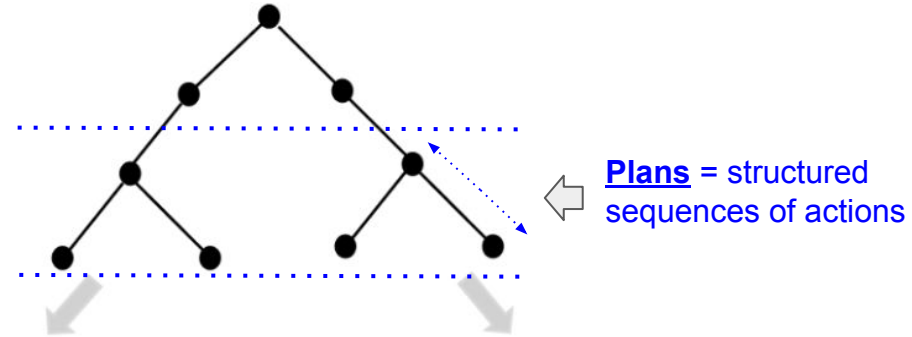
Information flows up and down “paths” in the hierarchy

Nodes = simpler problems given to specialists / units/ departments, etc.

Search is based on learning, and learning to select, **plans**



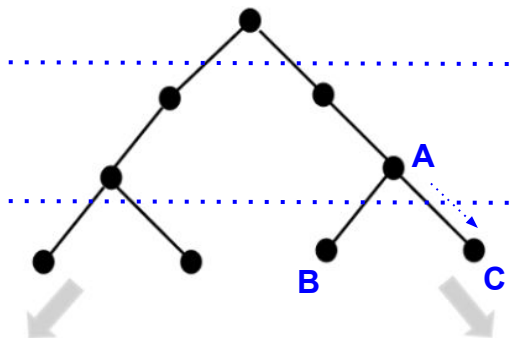
Hierarchical division of labor



*If a **plan** (sequence of actions) for solving problem is too complex, divide it into **simpler plans**.*

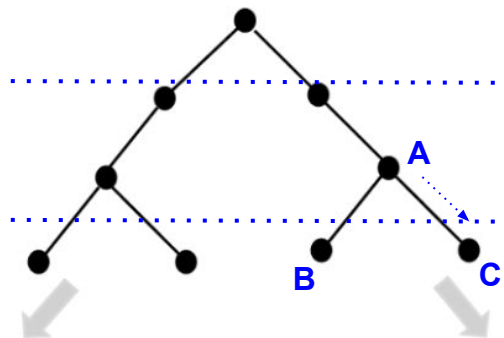
Planning is guided by **heuristics** (rules of thumb (stable beliefs) about which actions to take and when)

Original Problem



Example of a heuristic: “given I’m at node A, I should probably go to C rather than B.”

New, but Similar Problem



Plan for a similar problem might use the same heuristic. Even if the problem is not quite the same, the heuristic may be “**good enough**” (satisfactory)

Agents make plans (sequences of actions) to solve problems efficiently (economizing on information processing) by learning and using heuristics.

Search

Dividing the labor for solving a complex problem into **plans**, using **heuristics** to economize on information processing.

Search

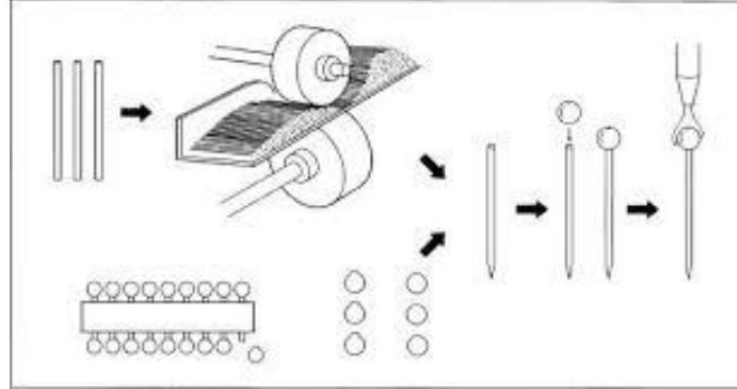
Dividing the labor for solving a complex problem into **plans**, using **heuristics** to economize on information processing.

Two approaches:

- Von Neumann search (playing a well-defined game)
- Simon search (structuring an ill-defined problem)

Plans: dividing the labor of problem-solving

Adam Smith's
(1776) pin factory



Tasks of (1) making pin bases; (2) making pinheads; and (3) assembling pins in a pin factory.

Division of labor = a strategy for learning

Dividing tasks among specialists enables **gains to specialization**. Why?

Simpler tasks = less interdependencies = easier to assign credit = better learning

****Not the same as the assembly line/mass production story**

Babbage's (1834) proposal: division of labor = a general solution to the credit assignment problem



(1) *Any* complex task can be divided into simpler subtasks.

(2) Given simple enough tasks, they can be mechanized.

(3) (1) and (2) can be applied to “thinking” tasks, not just physical ones such as pin-making. Hence, *any* thinking task can be divided up, learned and mechanized, and we can build a “**thinking machine**” that generally solves the credit assignment problem.

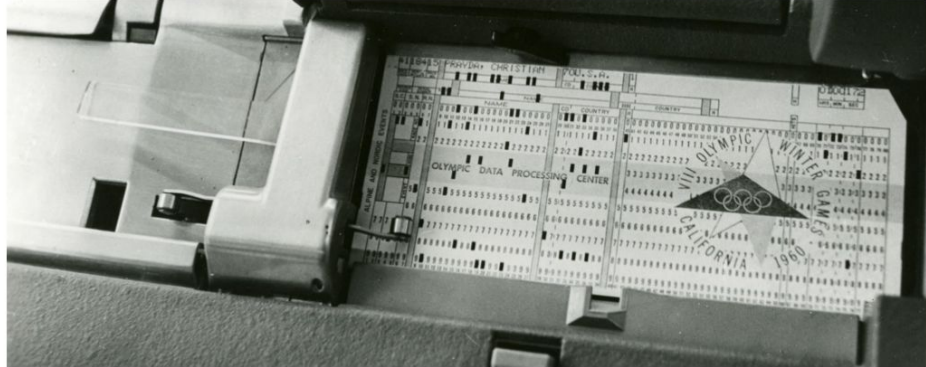
Dividing labor into plans (programs)

Plans mechanized as:

- **Algorithms:** sequences of instructions for actions.
- **Programs:** software + hardware for executing a plan (e.g., punched cards)



A program being inserted in a Jacquard loom for making a fabric pattern.



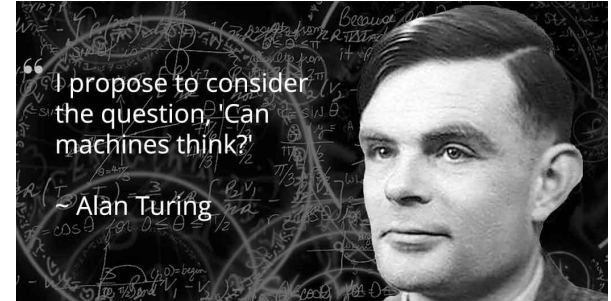
A punched card produced by IBM to store programs, and widely used in digital computers into the 1960s.

Effective procedures as measures of program complexity (i.e., procedural complexity)

Plans = guides for sequences of actions = step-by-step procedures

Program complexity = minimal effective procedure for solving a problem

Turing machines (1936) | Hypothetical machine with infinite memory (called a “tape”) to execute any program



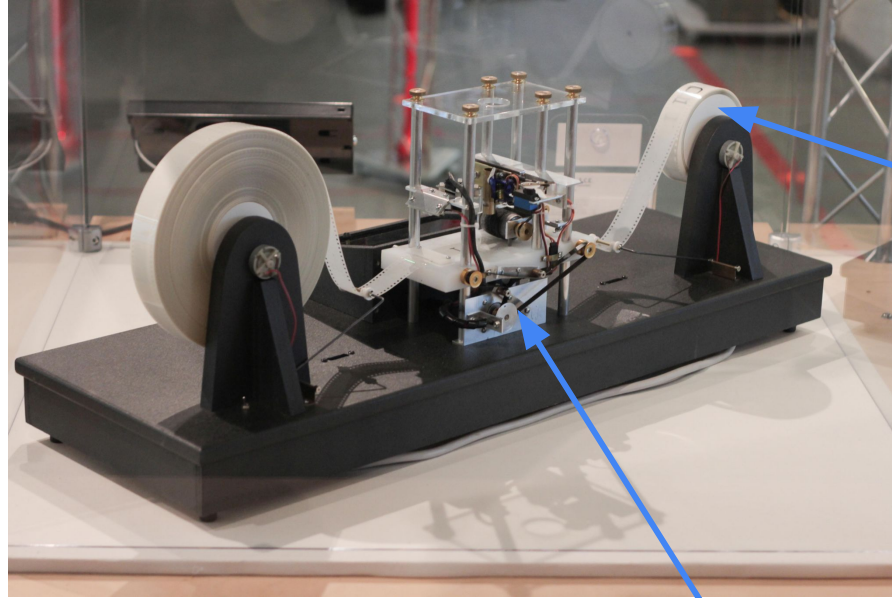
ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable numbers, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbersome technique. I hope

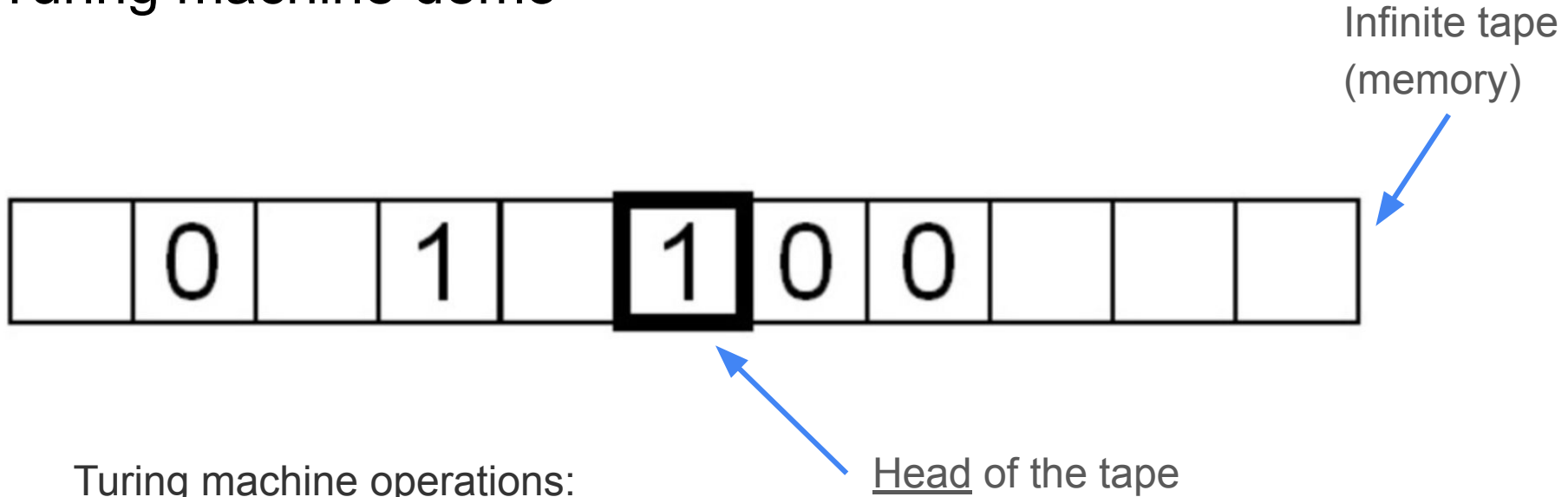
Turing machine demo (not really — tape should be infinite)



Infinite tape
(memory)

Head of the tape

Turing machine demo



Turing machine operations:

- (1) **Read** the symbol under the head;
- (2) **Edit** the symbol by transforming it into a new symbol or getting rid of it.
- (3) **Move** the head left or right.

***The symbols in a Turing machine are written as 0's and 1's for convenience. They need not be bits. They could be any symbols with any values. Program complexity depends on the level of abstraction at which symbols are written.*

As a trivial example to demonstrate these operations, let's try printing the symbols "1 1 0" on an initially blank tape:



First, we write a 1 on the square under the head:



Next, we move the tape left by one square:



Now, write a 1 on the new square under the head:



We then move the tape left by one square again:



Finally, write a 0 and that's it!



A simple program for a Turing machine

Symbol read	Write instruction	Move instruction
--------------------	--------------------------	-------------------------

Blank	None	None
-------	------	------

0	Write 1	Move tape to the right
---	---------	------------------------

1	Write 0	Move tape to the right
---	---------	------------------------

A simple program for a Turing machine - what's wrong?



The current symbol under the head is 0, so we write a 1 and move the tape right by one square.



The symbol being read is now 1, so we write a 0 and move the tape right by one square:



Similarly, the symbol read is a 1, so we repeat the same instructions.



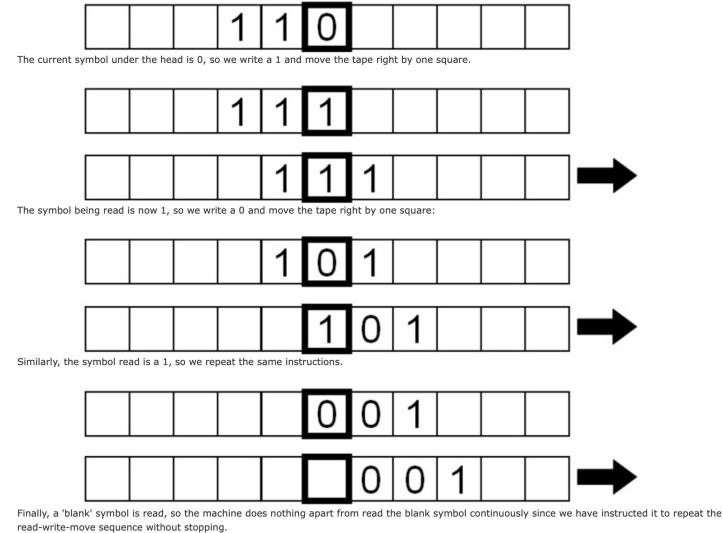
Finally, a 'blank' symbol is read, so the machine does nothing apart from read the blank symbol continuously since we have instructed it to repeat the read-write-move sequence without stopping.

Symbol read Write instruction Move instruction

Blank	None	None
0	Write 1	Move tape to the right
1	Write 0	Move tape to the right

States & transitions

State	Symbol read	Write instruction	Move instruction	Next state
<i>State 0</i>	Blank	None	None	<i>Stop state</i>
	0	Write 1	Move the tape to the right	<i>State 0</i>
	1	Write 0	Move the tape to the right	<i>State 0</i>

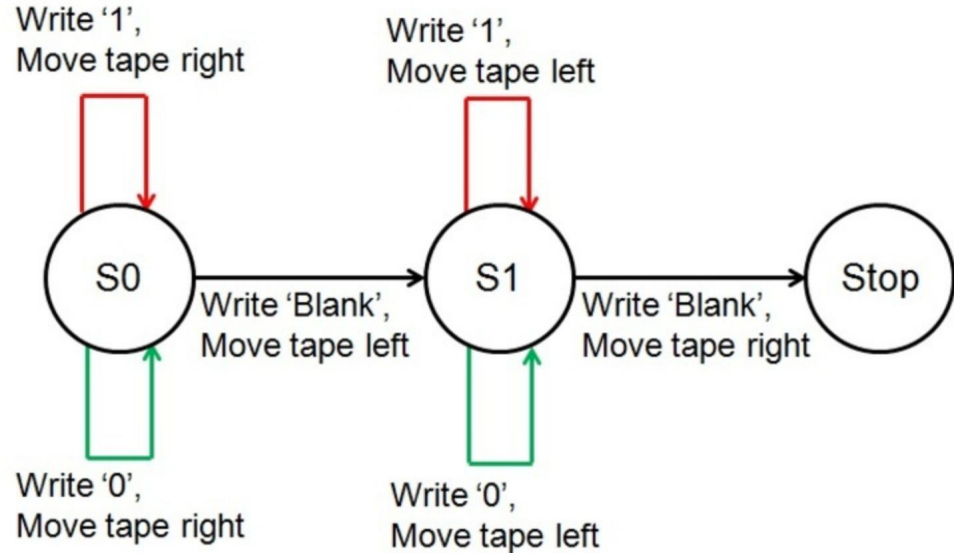


Finite state machines (e.g., a two-state Turing machine)

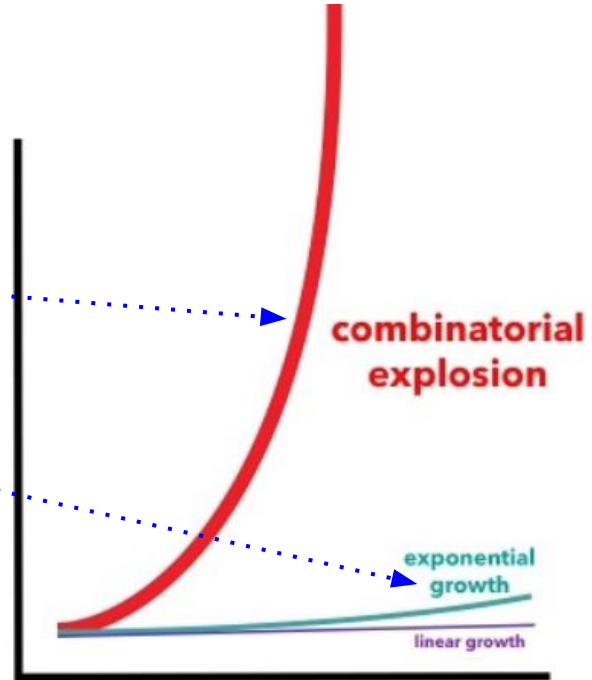
State	Symbol read	Write instruction	Move instruction	Next state
State 0	Blank	<i>Write blank</i>	<i>Move the tape to the left</i>	<i>State 1</i>
	0	Write 1	Move the tape to the right	State 1
	1	Write 0	Move the tape to the right	State 0
State 1	Blank	Write blank	Move the tape to the right	Stop state
	0	Write 1	Move the tape to the left	State 1
	1	Write 0	Move the tape to the left	State 1

Finite state machines (e.g., a two-state Turing machine)

State	Symbol read	Write instruction	Move instruction	Next state
State 0	Blank	Write blank	Move the tape to the left	State 1
	0	Write 1	Move the tape to the right	State 1
	1	Write 0	Move the tape to the right	State 0
State 1	Blank	Write blank	Move the tape to the right	Stop state
	0	Write 1	Move the tape to the left	State 1
	1	Write 0	Move the tape to the left	State 1



Program size can easily grow
combinatorially large — implies
Shannon entropy underestimates
actual complexity of problems.

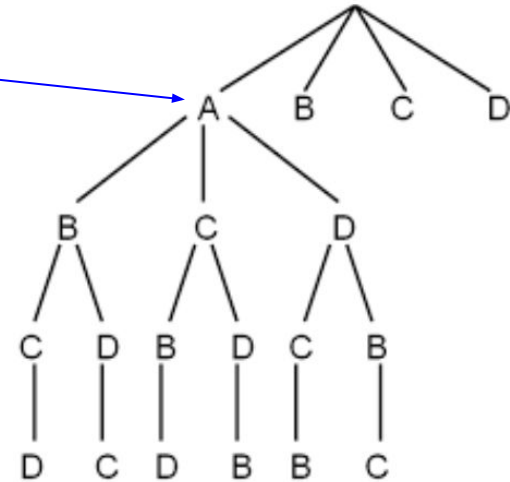


Search tree (or: search space/state space)

- (1) **State**: a description of a problem adequate to guide an agent to a goal (i.e., to trigger a heuristic for which action to take next)
- (2) **Initial state**: description of where the agent starts
- (3) **Transitions**: actions that take the agent to **intermediate states**
- (4) **Goal state**: description of some set of **satisfactory states**

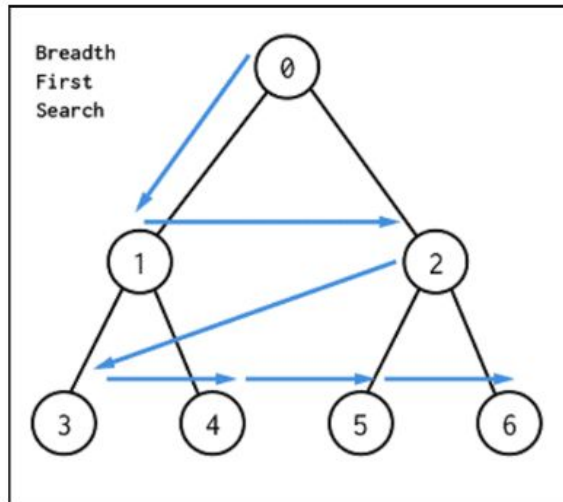
Search as a heuristic process: agents can only evaluate part of the search tree, hence must rely on heuristics

0	0	0	0	0	0
0	0	0	0	0	1
0	0	0	0	0	2
0	0	0	0	0	3

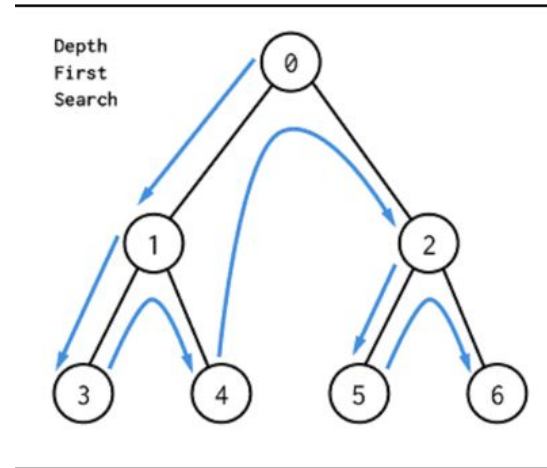


Heuristics may be general rules

Breadth-first heuristic | search many paths for a little bit



Depth-first heuristic | search one path, all the way through



Heuristics = if-then rules that are “**educated guesses**” to **economize** on **search costs**

Managers rely on many heuristics:

- (1) **Satisficing heuristics**: sticking with “good enough solutions”
(e.g., making this year’s budget/sales goal etc. to last year’s)
- (2) **Selection heuristics**: which product or market to enter
(e.g., “only invest in domestic markets”)
- (3) **Procedural heuristics**: how to enter a market, or perform a function
(e.g., “use M&A”; “use X market for both design and manufacturing”)

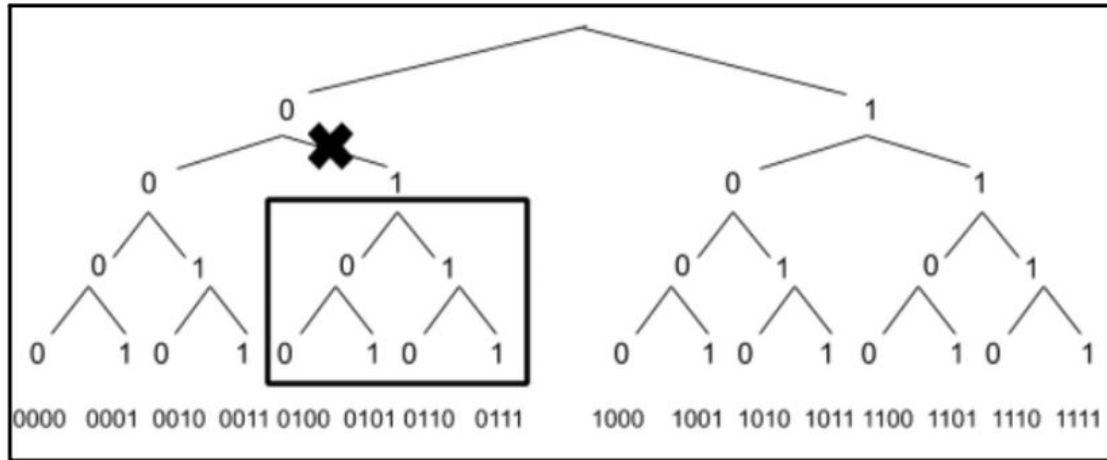
Heuristics may also be specific rules learned by agents (organizations develop “portfolios” of heuristics from experience to learn how to innovate)

Firm	Heuristic
F-Meddata	• Enter countries with lots of pharma activity • Target pharmaceutical companies • Promote electronic diary solutions to customers • Use standard criteria to qualify customers • Cold calling in host country by aggressive young Finn with a phone book • Hire local salespeople • Use direct sales approach • Ensure active communication among HQ and country employees through company intranet and email lists • Put more emphasis on the U.S. market than other markets • Place greater emphasis on getting Top 20 pharma reference accounts than other reference accounts • Move from small reference accounts to top 20 reference accounts • Qualify customers with template • Create success in major pharma markets (e.g., U.S. and Germany) before entering smaller markets (e.g., Japan)
F-Supplysoft	• Restrict internationalization to Scandinavia • Target wholesalers and independent retailers • Push enterprise software • Enter new countries through acquisitions (becomes elaborated in next entries) • Focus on three items in sales approach: reduce stock, control inventory, and improve replenishment process • Use direct sales to approach customers • Ensure pre-acquisition integration with target executives (becomes elaborated in next entries) • Hire a local country manager • Localize sales and marketing

	• Synchronize sales based on retail industry lifecycle • Stay more focused on Scandinavian markets than other markets • Create stepping-stones to larger markets (France, Germany, U.K.) • Place emphasis on grocery segment of retail markets • Choose countries with large domestic markets • Take one continent at a time (i.e., build Europe, move to U.S., then China)
U-Analytics	• Restrict internationalization to English-speaking markets • Target large enterprises • Sell real-time analytics • Focus on a features selling approach that highlights detail and functions of technology • Use direct sales • Work with experienced local country manager
	• Use implementation partners • Create a strong HQ liaison for each country • Use U.K. as launching pad into large markets in continental Europe • Solidify the organization in the current entry, then enter the next • Emphasize low costs serendipitous mode to enter new countries over other higher cost modes (e.g., greenfield, acquisition)
U-Semi	• Enter countries that have original device manufacturers (ODMs) and original equipment manufacturers (OEMs) • Sell turnkey semiconductor solutions

Firm	Heuristic
	• Use China for both design and manufacturing of chips • Use consultant to provide insight about local market and develop relationships with distributors • Use direct sales • Emphasize low cost • In the sales pitch, emphasize track record of CEO (elaborated during later entries) • Create partnerships with other chipset and software providers in order to supply a total solution • Focus efforts on tier-one countries versus other tiers • Move from tier-three to tier-two to tier-one countries • Within the tier-three and tier-two countries, develop and validate products in Taiwan first, then in Korea, and then take them to China • Synchronize sales approach to local culture • Start first with auto, then PDA, then mobile handset • Pace new chip features according to one-year auto industry design cycle
S-Enterprise	• Stay Asia focused • Promote B2B e-collaboration software • Target large, multinational OEMs • Leverage use of standards body when entering new countries • Use greenfield entry mode • Have a TMT member oversee country operations (i.e., be in focal country) • Use indirect sales that relies on partners • Work closely with partner on first deal • Put greatest emphasis on Japanese market within Asia • Put more emphasis on Asian countries w/Rosetta Net • Create business case for entry into country

Heuristics allow “**pruning**” the search tree (eliminating the need to assign credit to entire “branches”, economizing on search costs).



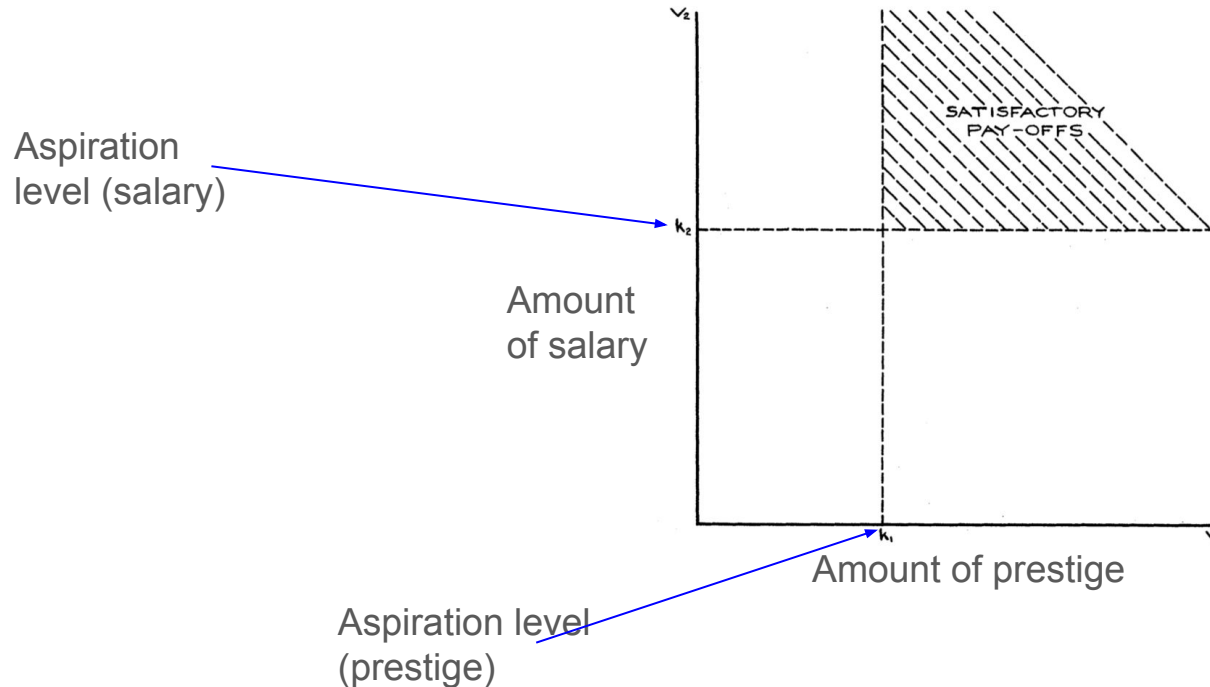
Satisficing

Any use of heuristics = not evaluating all nodes of the search tree (no brute force).

Hence, in search, no guarantee of an “optimal” plan/solution to a problem; the most one can expect is a “**satisfactory**” solution

Why satisficing is necessary to assign credit intelligently

(1) **Economics explanation** | Search costs outweigh benefits of more search above a threshold (“aspiration level”).



Why satisficing is necessary to assign credit intelligently

(2) Information processing explanation | Search is based on symbols that merely designate a vast physical world. Hence, search is always based on a “small world” simplification of “large world” reality. .



Small Worlds



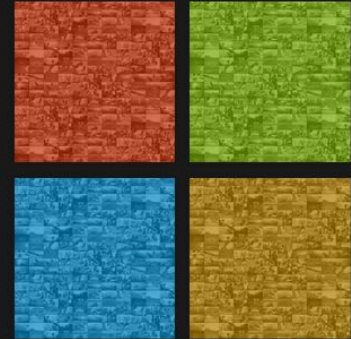
Large Worlds

Why satisficing is necessary to assign credit intelligently

(2) Organizational explanation | Orgs. have goals that are multiple, contradictory, and qualitative in nature. Search is based on plans that are really organizational programs.

Microsoft mission

Empower every person and
every organization on the
planet to achieve more



Search processes (review)

Search = heuristic process of generating plans until a satisfactory one is found

Benefit of search = efficiency (pruning the search tree/state space)

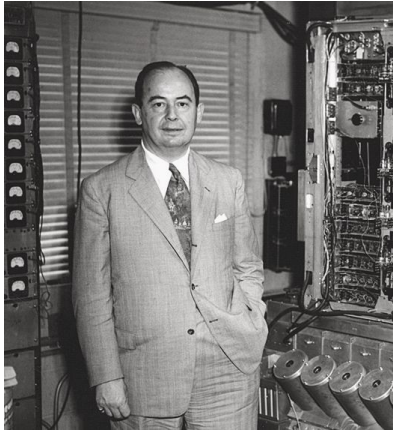
Yet needs assumptions: (1) What is a state? (2) How do agents evaluate them?

Two approaches:

- Von Neumann search: extends reinf. learning paradigm, Shannon information
- Simon search: extends learning-in-a-society paradigm, Simon info.

(1) Von Neumann search | search as playing a well-defined game

(1) Von Neumann search | search as playing a well-defined game



John von Neumann
(1903-1957)

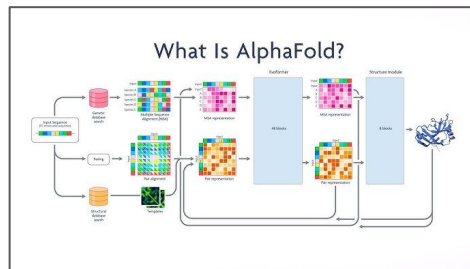
Games (poker, chess, etc.) = problems with:

- Two or more agents.
- Well-defined rules.
- Other agent's actions can be hostile.

The game hypothesis: if a search program can play Go... it will soon learn to search for a solution to any problem.



AlphaGo vs. Lee Sedol
(2016): Game of Go



AlphaFold (2018):
Protein structures



Artificial General
Intelligence?

Assumptions of the game hypothesis

- (1) Focuses on search **algorithms**, not implementing search programs (ultimately, relies on a lot of brute force)
- (2) **Unitary actor assumption**: focuses on mathematics; little discussion of the division of labor in orgs. and the real-world challenges that presents. Assumes a unitary actor (i.e., a CEO) controls the whole search process.

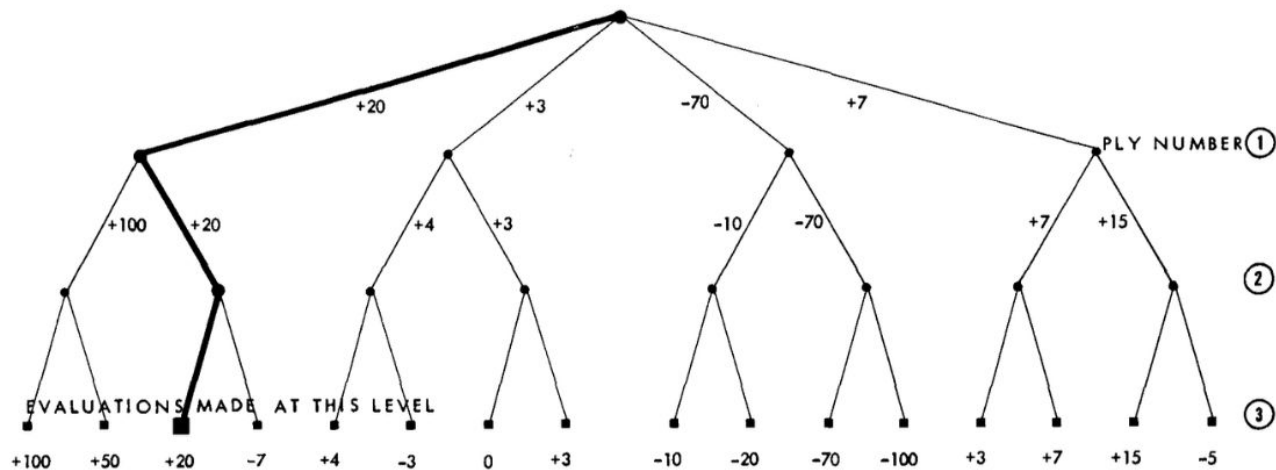
Heuristics are defined as economic utility functions

Assumes well-defined games, hence no need for physical symbols to describe all information about the game (states, goals, actions, etc.)

Solutions to problems = utility = a single number = like RL rewards

Example: Samuel's 1959 checkers program. Player 1 = "Max"; Player 2 = "Min"

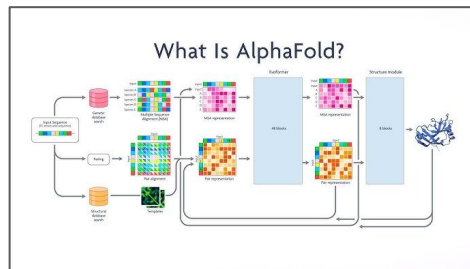
- Credit assignment is based on numerical values ("utility") for each state, calculated using some "**evaluation function**" pieced together from various intermediate feedback (how many pieces left, how many kings left, etc.).
- Heuristics = **numerical patterns** learned from trial-and-error about positions on the board



The game hypothesis: if a search program can play Go... it will soon learn to search for a solution to any problem.



AlphaGo vs. Lee Sedol
(2016): Game of Go



AlphaFold (2018):
Protein structures



?

Artificial General
Intelligence?

Let us refine the “game hypothesis”: it is a bit like a messier version of Sutton’s reward hypothesis

Any problem can be learned using some (messy) **evaluation function** (hence, search & satisficing) + **reinf. learning** (hence, brute force & maximizing rewards)



(2) Simon search | structuring ill-defined problems

(1) Simon search | Structuring ill-defined problems



Allen Newell and Herbert Simon (1959)

Ill-defined problems = problems with:

- Loosely coordinated agents.
- Ill-defined or evolving rules.
- Goals are ambiguous, shifting, or constructed during search
- Other agent's actions are not explicitly hostile.

General problem-solver (GPS) (1959)

Actions =

- Evaluation (matching patterns in current state vs. goal state (ends))
- Selecting and executing operators (means) to transform symbols

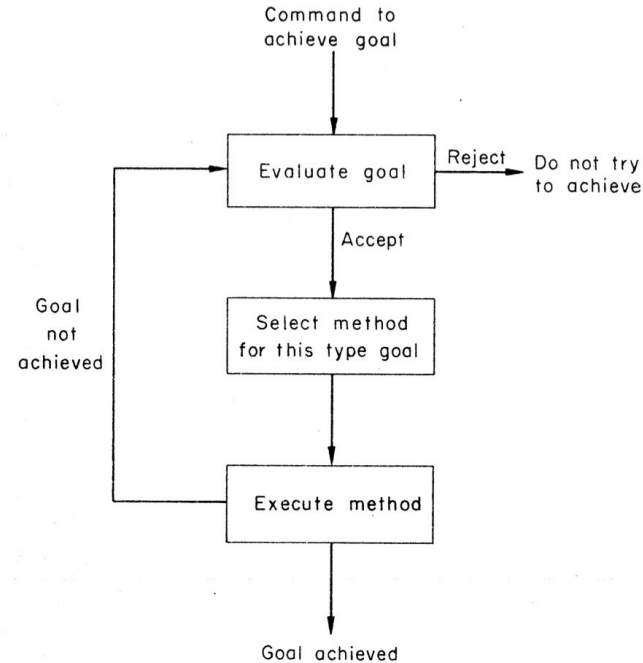
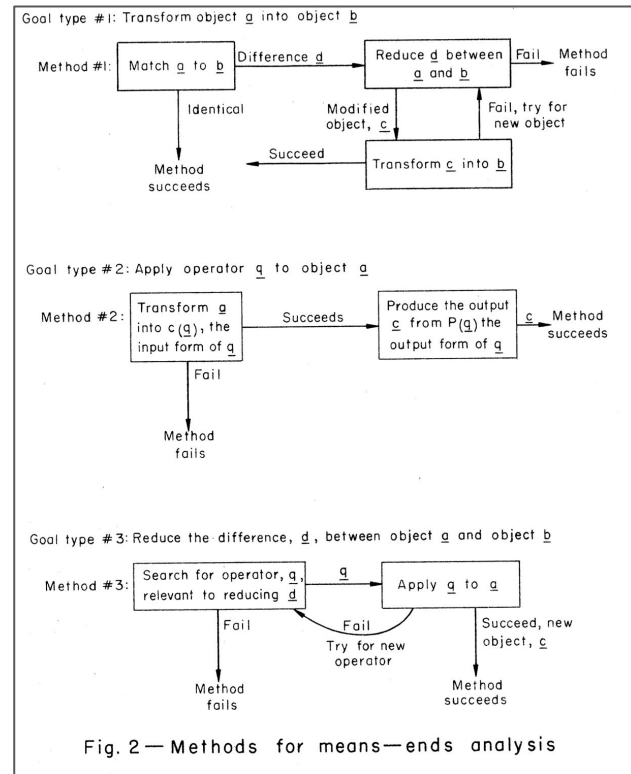


Fig. 1—Executive organization of GPS

General Problem Solver (GPS) (1959)

Heuristics = general rules for selecting operators (means) to transform symbols given a state closer to the goal state (ends)

- Search = using and revising plans to select operators for reducing the difference between some current state and the goal state
- **All** based on **physical symbols** — no measure of utility/rewards needed



Simon search | “means-ends analysis”

States = objects = physical symbols

Goal states and **subgoal states** = “ends”

Operators = “means”, the set of actions for transforming objects, with well-defined rules (e.g., legal chess moves)

Means-ends analysis:

(1) measure difference between current state and goal state → (2) select a heuristic to reduce the difference; (3) if a difference still exists, create a subgoal.

Means-ends analysis example: solve $5x + 3 = 2X + 7$

States = objects in the algebra equation

Goal state = structuring the algebra equation so that it is trivial to solve

Operators = addition, subtraction, multiplication, grouping, etc.

Means-ends heuristic:

(1) measure difference between current equation and goal equation → (2) use one of the operators to reduce the difference; (3) if a difference still exists, create a new subgoal of reducing the difference further, and select a new operator

Means-ends analysis example: solve $5x + 3 = 2X + 7$

1. Subtract $2x$ from both sides:

$$5x + 3 - 2x = 2x + 7 - 2x$$

This simplifies to:

$$3x + 3 = 7$$

2. Subtract 3 from both sides:

$$3x + 3 - 3 = 7 - 3$$

This simplifies to:

$$3x = 4$$

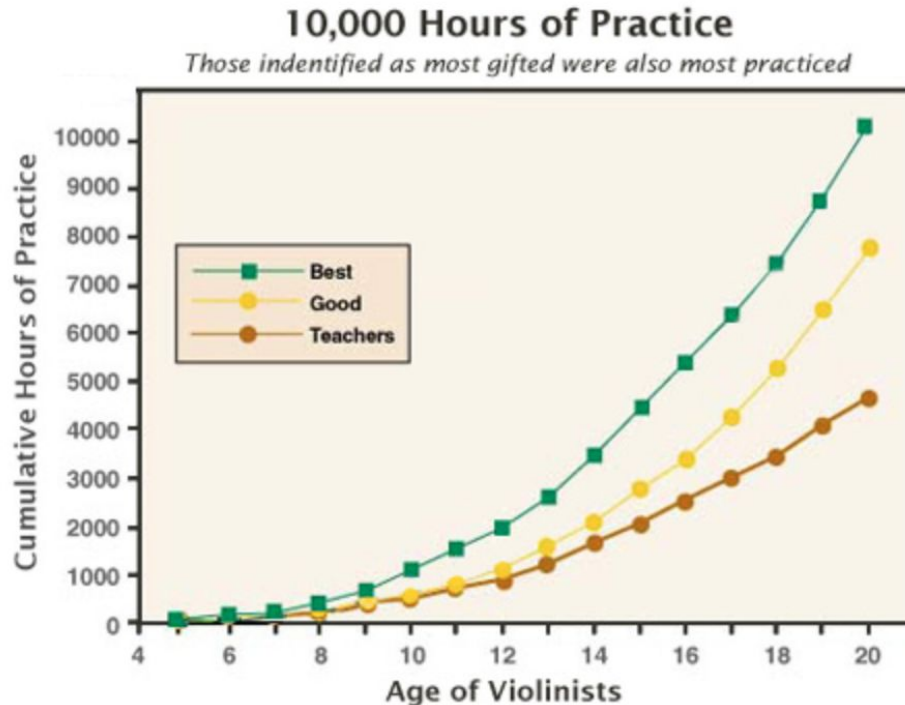
3. Divide both sides by 3:

$$x = \frac{4}{3}$$

Means-ends analysis — how did it work?

- (1) **Pattern matching** | evaluated each symbol for whether it matched the goal state (i.e., an algebra equation with all X variables on one side)
- (2) **Generate-and-test** | if the pattern does not match, create a subgoal and select an operator (add, divide, etc.) to reduce the difference
- (3) **Acquire heuristics** | as one gains experience solving algebra problems, one knows how to efficiently select heuristics (operators) given any difference

“Study experts” hypothesis: if we know how experts use heuristics to learn, then we can build AI systems for any problem



Example | 10,000 hour rule for acquiring expertise.

Search view | difference in expertise = difference in amount of heuristics learned